Technical Documentation

CV-Form-Filler - Arquitectura interna, modulos y flujos

Documentacion tecnica y de usuario

Herramienta 08 - Coleccion Scripts_Ciberseguridad

Version 1.0.0 - Noviembre de 2026

Autor: Danny-Developer2

Basado en el codigo real de la version 1.0.0

Uso interno - contiene datos de ejemplo. No distribuir sin autorizacion.

Contenido

1. Introduccion . . . 3
2. Estructura del proyecto . . . 3
3. Arquitectura interna . . . 3
4. Modelos de datos . . . 3
4.1 Definicion de modelos . . . 4
5. Modulos . . . 5
5.1 cli.py . . . 5
5.2 cv_parser.py . . . 5
5.3 form_inspector.py . . . 5
5.4 mapper.py . . . 5
5.5 filler.py . . . 6
5.6 verifier.py . . . 6
5.7 utils/logger.py . . . 6
5.8 exceptions.py . . . 6
6. Motor de mapeo . . . 6
6.1 Normalizacion . . . 6
6.2 Similitud . . . 6
6.3 Logica de tablas . . . 7
6.4 Campos faltantes . . . 7
7. Flujos . . . 7
7.1 Flujo normal . . . 7
7.2 Flujo dry-run . . . 7
7.3 Flujo de error . . . 7
8. Configuracion . . . 7
8.1 Dependencias declaradas . . . 7
8.2 Opciones de la CLI . . . 8
8.3 Configuracion de pytest, ruff y mypy . . . 8
9. Manejo de errores . . . 8
10. Formato de resultados . . . 9
10.1 FillReport . . . 9
10.2 VerificationReport . . . 9
10.3 Informe JSON de ejemplo . . . 9
10.4 Tabla de la CLI . . . 9
11. Verificacion y pruebas . . . 10

1. Introduccion

Este documento describe la arquitectura interna de CV-Form-Filler, sus modulos, modelos de datos, motor de mapeo, flujos de ejecucion, configuracion, manejo de errores y formato de resultados. Esta basado en el codigo real de la version 1.0.0.

2. Estructura del proyecto

```
08-CV-Form-Filler/
|-- main.py                # punto de entrada python main.py
|-- pyproject.toml         # dependencias, ruff, mypy, pytest
|-- Makefile               # install, test, coverage, lint, format, types, check
|-- requirements.txt       # dependencias declaradas
|-- src/cvformfiller/
|   |-- __init__.py        # version y metadatos
|   |-- __main__.py        # python -m cvformfiller
|   |-- cli.py             # CLI Typer + Rich
|   |-- models.py          # modelos Pydantic inmutables
|   |-- exceptions.py      # jerarquia de excepciones
|   |-- cv_parser.py       # CV PDF/DOCX -> CvData
|   |-- form_inspector.py  # PDF -> list[FormField]
|   |-- mapper.py          # CvData + FormField[] -> MappingResult
|   |-- filler.py          # MappingResult -> PDF
|   |-- verifier.py        # PDF -> VerificationReport
|   `-- utils/logger.py    # logging sin datos personales
|-- tests/
|   |-- conftest.py        # fixtures de sesion
|   |-- fixtures/          # CV y formulario de prueba
|   |-- unit/              # pruebas unitarias por modulo
|   `-- integration/       # pruebas de la CLI
|-- docs/                  # idea.md, arquitectura.md y PDFs
|-- examples/              # ejemplos JSON y generadores
`-- reports/               # salidas generadas (ignoradas por git)
```

3. Arquitectura interna

La pipeline tiene cinco etapas. Cada una es un modulo con una responsabilidad unica y devuelve un modelo inmutable. La CLI orquesta el flujo con una barra de progreso y mostrando el resultado final.

```
cv_parser.parse_cv(path) -> CvData
form_inspector.inspect_form(path) -> list[FormField]
mapper.map_cv_to_form(cv, fields) -> MappingResult
filler.fill_form(form, fields, mapping, output, ...) -> FillReport
verifier.verify_fill(output, fields, mapping) -> VerificationReport
```

Etapa	Responsabilidad	Dependencias
cv_parser	Extraer texto del CV y normalizarlo en CvData.	pdfplumber, pydantic
form_inspector	Leer AcroForm y localizar etiquetas por geometria.	pypdf, pdfplumber
mapper	Asignar conceptos del CV a campos del formulario.	Solo modelos
filler	Escribir una copia del PDF con valores y apariciones.	pypdf
verifier	Comprobar valores /V y el renderizado.	pypdf, pypdfium2

4. Modelos de datos

Todos los modelos son inmutables (ConfigDict(frozen=True)), lo que garantiza que el CV parseado y el mapeo no se modifiquen una vez construidos.

Modelo	Descripcion	Campos principales
FieldType	StrEnum de tipo de campo.	TEXT, CHECKBOX, RADIO, UNKNOWN

Modelo	Descripcion	Campos principales
Contacto	Datos de contacto.	telefono, email, web
Experience	Entrada de experiencia laboral.	puesto, empresa, periodo, descripcion
Education	Entrada de formacion academica.	titulo, institucion, periodo, grado
CvData	CV normalizado.	nombre, apellidos, contacto, fecha_nacimiento, domicilio, ciudad, estado, nacionalidad, curp, rfc, nss, experiencia, educacion, idiomas, habilidades, software
FormField	Campo del formulario.	name, type, rect, page, label, row_label, column_label, confidence
MappedField	Campo con el valor asignado.	field, label, value, source, confidence
MappingResult	Resultado del mapeo.	fields, missing, total_fields, filled_count, fill_ratio
FillReport	Informe del rellenado.	output, total_fields, fields_filled, fields_missing, missing_fields, font_size, dry_run, checks_passed, checks_failed, errors, duration_seconds
VerificationReport	Resultado de la verificacion.	ok, checks_passed, checks_failed, rendered_pages, errors

4.1 Definicion de modelos

```
class FieldType(StrEnum):
    TEXT = "text"
    CHECKBOX = "checkbox"
    RADIO = "radio"
    UNKNOWN = "unknown"

class CvData(BaseModel):
    model_config = ConfigDict(frozen=True)
    nombre: str = ""
    apellidos: str = ""
    contacto: Contacto = Field(default_factory=Contacto)
    fecha_nacimiento: str = ""
    domicilio: str = ""
    ciudad: str = ""
    estado: str = ""
    nacionalidad: str = ""
    curp: str = ""
    rfc: str = ""
    nss: str = ""
    experiencia: list[Experience] = Field(default_factory=list)
    educacion: list[Education] = Field(default_factory=list)
    idiomas: list[str] = Field(default_factory=list)
    habilidades: list[str] = Field(default_factory=list)
    software: list[str] = Field(default_factory=list)

class FormField(BaseModel):
    model_config = ConfigDict(frozen=True)
    name: str
    type: FieldType
    rect: tuple[float, float, float, float]
    page: int
    label: str = ""
    row_label: str = ""
    column_label: str = ""
    confidence: float = 0.0
```

5. Modulos

5.1 cli.py

Define un Typer con un callback principal para que las opciones funcionen directamente (python main.py --cv ... --form ... --output ...). Muestra un banner, el aviso de privacidad, una barra de progreso y la tabla de campos rellenados y faltantes. Codigos de salida: 0 exito, 1 error, 2 opciones incompletas y 130 interrupcion.

```
@app.callback(invoke_without_command=True)
def run(ctx, cv, form, output, font_size=10.0, dry_run=False, verbose=False, log_file=None):
    if ctx.invoked_subcommand is not None:
        return
    if cv is None or form is None or output is None:
        err_console.print("Error: son obligatorios --cv, --form y --output.")
        raise typer.Exit(code=2)
    setup_logger("DEBUG" if verbose else "INFO", log_file)
    cv_data, campos, mapping, report = _run(cv, form, output, font_size, dry_run)
    _print_summary(cv_data.nombre, campos, mapping, report)
```

5.2 cv_parser.py

parse_cv(path) elige el extractor segun la extension: PDF con pdfplumber o DOCX con zipfile y xml.etree. El texto se agrupa por secciones y se normaliza en CvData. El parser es heuristico y no registra datos personales en el log.

Patron / heuristica	Descripcion
_EMAIL_RE	Correo electronico
_PHONE_RE	Telefono; descarta rangos de anos 19xx-20xx
_CURP_RE	CURP mexicano
_RFC_RE	RFC mexicano
_NSS_RE	Numero de seguridad social
_BIRTH_DATE_RE	Fecha de nacimiento en tres formatos
_CITY_STATE_RE	Ciudad y estado en el encabezado
_STATE_EXPANSIONS	Expansion de abreviaturas de estado
_SOFTWARE_KEYWORDS	Tecnologias y herramientas conocidas
_KNOWN_HEADERS	Encabezados de seccion reconocidos

5.3 form_inspector.py

Lee el diccionario /AcroForm con pypdf y recorre /Fields. Para cada campo registra nombre, tipo, rectangulo y pagina. Extrae las palabras con pdfplumber y busca la etiqueta mas cercana por proximidad geometrica.

Constante	Valor	Descripcion
_WORD_GAP	25.0	Separacion maxima entre palabras de una etiqueta
_LINE_TOLERANCE	2.5	Tolerancia vertical para agrupar en la misma linea
_RADIO_BIT	1 << 15	Bit /Ff que identifica un boton de radio

Busqueda de etiquetas: encima o debajo (solape horizontal y distancia vertical minima, agrupando palabras contiguas) y a la izquierda (bloques de varias lineas, por ejemplo "Nombre de la Compania").

5.4 mapper.py

Asigna conceptos del CV a campos del formulario. La funcion principal es map_cv_to_form(cv, fields), que ejecuta en orden: experiencia, escolaridad, estudios actuales y conceptos simples.

5.5 filler.py

Lee el formulario con pypdf, lo clona en un PdfWriter y actualiza los valores /V regenerando las apariciones con update_page_form_field_values. Los campos de texto usan la tupla (valor, "/Helv", font_size); las casillas usan /Yes. Nunca sobrescribe el original.

5.6 verifier.py

Lee el PDF de salida con pypdf y comprueba que cada /V coincide con el valor mapeado (texto) o con /Yes (casillas). Despues renderiza con pypdfium2 y comprueba que la region de cada campo rellenado contiene pixeles oscuros.

Constante	Valor	Descripcion
_RENDER_SCALE	2.0	Escala de renderizado
_DARK_THRESHOLD	128	Umbral de pixel oscuro en escala de grises
_MIN_DARK_RATIO	0.002	Ratio minimo de pixeles oscuros para considerar dibujado

5.7 utils/logger.py

Configura el logger cvformfiller con salida a stderr y, opcionalmente, a un fichero. Los datos personales nunca se escriben en el registro: solo nombres de campo, contadores y rutas tecnicas.

5.8 exceptions.py

Jerarquia de excepciones tipadas que no exponen valores del CV: CvFormFillerError como base, con CvParseError, FormInspectionError, MappingError, FillError y VerificationError.

6. Motor de mapeo

El mapper combina tres heuristicas: normalizacion y similitud de etiquetas, proximidad geometrica ya resuelta por el inspector y posicion/orden dentro de tablas repetidas.

6.1 Normalizacion

```
def _normalize(text: str) -> str:
    value = text.lower()
    value = unicodedata.normalize("NFD", value)
    value = "".join(c for c in value if not unicodedata.combining(c))
    value = value.replace("(", "").replace(")", "")
    value = re.sub(r"[^a-z0-9]+", " ", value)
    return re.sub(r"\s+", " ", value).strip()
```

El resultado es una cadena en minusculas, sin acentos, sin puntuacion y con una sola separacion entre tokens.

6.2 Similitud

Caso	Puntuacion
Igualdad exacta tras normalizar	1.0
La etiqueta empieza por el alias	0.9
El alias es un token de la etiqueta	0.85
El alias esta contenido en la etiqueta	0.7
Tokens comunes (0.5 x comunes / alias)	0.0 - 0.5
Sin coincidencia	0.0

Cada campo puntua contra cada alias con cuatro variantes ponderadas: etiqueta completa (1.0), etiqueta de columna (0.9), etiqueta de fila (0.6) y etiqueta combinada (0.5). Se asigna un concepto como maximo y los campos ya usados no se reutilizan.

6.3 Logica de tablas

Tabla	Estrategia
Experiencia	Campos de compania, puesto y periodo (de/a) ordenados por posicion X; se rellenan hasta tres empleos.
Escolaridad	Filas por nivel (primaria, secundaria, preparatoria, profesional, comercial) y columnas por etiqueta o posicion (nombre, direccion, de, a, anos, titulo).
Estudios actuales	Escuela y curso/carrera del estudio mas reciente o en curso.

6.4 Campos faltantes

Un campo se registra en missing cuando el mapper lo identifica como relevante (por regla o por tabla) pero el CV no aporta un valor. No es el total de campos vacios del formulario, sino el conjunto de campos que la herramienta sabe interpretar y no pudieron rellenarse.

7. Flujos

7.1 Flujo normal

1. parse_cv(cv) -> CvData 2. inspect_form(form) -> list[FormField] 3. map_cv_to_form(cv, f) -> MappingResult 4. fill_form(...) -> FillReport + PDF de salida 5. verify_fill(output) -> VerificationReport 6. CLI imprime resumen y ruta del PDF

7.2 Flujo dry-run

1. parse_cv(cv) -> CvData 2. inspect_form(form) -> list[FormField] 3. map_cv_to_form(cv, f) -> MappingResult 4. fill_form(dry_run=True) -> FillReport (sin escribir PDF) 5. CLI muestra el mapeo y confirma que no se escribio ningun PDF

7.3 Flujo de error

Excepcion tipada (CvParseError, FormInspectionError, FillError, ...) -> cli.py la captura -> mensaje sin valores del CV en stderr -> typer.Exit(code=1) KeyboardInterrupt -> mensaje de interrupcion y typer.Exit(code=130)

8. Configuracion

8.1 Dependencias declaradas

Dependencia	Rango	Uso
pypdf	>=5.0,<7	Lectura/escritura de AcroForm y valores /V
pdfplumber	>=0.11,<1	Extraccion de texto y palabras con coordenadas
pypdfium2	>=4.30,<6	Renderizado del PDF para verificar el dibujado
pydantic	>=2.10,<3	Modelos de datos inmutables validados
typer	>=0.15,<1	Interfaz de linea de comandos
rich	>=13.9,<14	Banner, tablas y barra de progreso en consola
Jinja2	>=3.1,<4	Motor de plantillas (dependencia transitiva declarada)

8.2 Opciones de la CLI

Opcion	Alias	Default	Descripcion
--cv	-c	obligatorio	CV en PDF o DOCX
--form	-f	obligatorio	Formulario PDF con campos AcroForm
--output	-o	obligatorio	PDF de salida
--font-size		10.0	Tamano de fuente uniforme (pt)
--dry-run		False	Solo muestra el mapeo sin escribir el PDF
--verbose	-v	False	Registro detallado (DEBUG)
--log-file		None	Volcar el registro a un fichero

8.3 Configuracion de pytest, ruff y mypy

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q --strict-markers"

[tool.ruff]
line-length = 100
target-version = "py311"

[tool.mypy]
python_version = "3.11"
strict = true
```

9. Manejo de errores

Todas las excepciones derivan de CvFormFillerError, lo que permite a la CLI capturar cualquier fallo interno con un solo bloque. Los mensajes no incluyen valores del CV.

Situacion	Excepcion	Codigo de salida
CV inexistente o formato no admitido	CvParseError	1
CV vacio o sin capa de texto	CvParseError	1
Formulario sin AcroForm	FormInspectionError	1
No se puede escribir la salida	FillError	1
La salida es el propio formulario original	FillError	1
Tamano de fuente <= 0	FillError	1
Verificacion con errores	VerificationError (reporte)	0 con avisos
Opciones incompletas	typer.Exit	2
Interrupcion del usuario	KeyboardInterrupt	130

10. Formato de resultados

10.1 FillReport

Campo	Tipo	Descripcion
output	str	Ruta del PDF generado
total_fields	int	Campos del formulario
fields_filled	int	Campos con valor
fields_missing	int	Campos relevantes sin datos
missing_fields	list[str]	Nombres de campo faltantes
font_size	float	Tamano de fuente aplicado
dry_run	bool	Indica si fue una ejecucion sin escritura
checks_passed	int	Comprobaciones de verificacion superadas
checks_failed	int	Comprobaciones fallidas
errors	list[str]	Errores de verificacion
duration_seconds	float	Duracion de la operacion

10.2 VerificationReport

Campo	Tipo	Descripcion
ok	bool	Verdadero si no hubo fallos
checks_passed	int	Comprobaciones superadas
checks_failed	int	Comprobaciones fallidas
rendered_pages	int	Paginas renderizadas con pypdfium2
errors	list[str]	Detalle de los fallos

10.3 Informe JSON de ejemplo

El script examples/generar_informe.py produce un informe JSON con el candidato, el conteo de campos, el mapeo completo y la verificacion. Fragmento real:

```
{
  "candidato": "Juan Perez",
  "campos_formulario": 222,
  "campos_rellenados": 39,
  "campos_faltantes": ["Text111", "Text113", "..."],
  "mapeo": [
    {"campo": "Text10", "etiqueta": "Apellido Paterno",
     "valor": "Perez", "origen": "apellido_paterno"},
    {"campo": "Text98", "etiqueta": "Profesional",
     "valor": "Ingeniería Ficticia", "origen": "escolaridad_nombre"}
  ],
  "verificacion": {"ok": true, "comprobaciones_pasadas": 40,
                   "comprobaciones_fallidas": 0, "paginas_renderizadas": 2},
  "salida": "reports/solicitud_rellenada.pdf"
}
```

10.4 Tabla de la CLI

Columna	Descripcion
Campo	Nombre interno del campo AcroForm
Etiqueta	Etiqueta impresa localizada por proximidad
Valor	Valor asignado desde el CV
Origen	Concepto del CV que alimento el campo

11. Verificacion y pruebas

La suite de pruebas cubre modelos, parser, inspector, mapper, filler, verifier y la CLI. El detalle de los 77 casos y la cobertura del 93 % se documenta en QA_TEST_REPORT.pdf.